

# Análisis de la curva de carga y descarga de un condensador mediante adquisición de datos con una placa FPGA y comparación con la teoría de circuitos RC

Ana María Hurtado\*, Ana Sofía Mora†, Estiven Castrillon‡

Instituto de Física, Universidad de Antioquia

Laboratorio Avanzado 1 - Junio 2025

\*ana.hurtador@udea.edu.co

†ana.mora@udea.edu.co

‡estiven.castrillon2@udea.edu.co

**Resumen**—En esta práctica se implementó un sistema de adquisición de datos basado en una FPGA (Field Programmable Gate Array) para registrar la curva de carga y descarga de un condensador, tal que los datos fueron transferidos al computador mediante DMA (Direct Memory Access) y procesados con Python. Los resultados experimentales muestran el comportamiento exponencial característico de los circuitos RC, con un voltaje máximo normalizado de aproximadamente  $0,176 V/V_{max}$ , consistente con la teoría clásica de circuitos, aquí se comparan los valores experimentales con los predichos por el modelo teórico y se discuten las posibles fuentes de error.

## I. INTRODUCCIÓN

Los circuitos RC (resistencia-condensador) constituyen uno de los sistemas más fundamentales en la electrónica analógica, y en la física experimental son el ejemplo por excelencia para experimentos simples de electrónica por su facilidad de construcción y estudio. Su dinámica temporal está gobernada por la constante de tiempo  $\tau = RC$  [1, p. 274, eq. (16)], que determina la rapidez con que el condensador se carga o descarga ante una excitación escalón. Desde el punto de vista experimental, la medición de estas curvas requiere instrumentación adecuada; en años recientes, las FPGAs han ganado lugar como plataformas de adquisición de datos en laboratorio gracias a su capacidad de operar en tiempo real con alta precisión temporal y reconfigurabilidad.

Una FPGA es un circuito integrado que contiene bloques lógicos programables interconectados, cuya funcionalidad puede definirse mediante lenguajes de descripción de hardware como VHDL o Verilog [2]. A diferencia de un microprocesador, la FPGA ejecuta operaciones en paralelo a nivel de hardware, lo que la hace especialmente adecuada para la captura de señales analógicas a través de conversores ADC (Analog to Digital Converter).

En este trabajo se utilizó la tarjeta PYNQ-Z1, una plataforma que integra una FPGA Zynq de Xilinx con un procesador ARM, programable desde Python mediante la librería `pynq` [3]. Esta combinación permite implementar el hardware de ad-

quisición directamente en la FPGA y acceder a los datos desde el entorno de Python sin necesidad de drivers adicionales.

## II. MARCO TEÓRICO

### II-A. Carga y descarga de un condensador

Si consideramos un circuito RC, (ver sección III-B), como el que se plantea en la metodología, cuando se aplica una tensión  $V_0$  al circuito, la corriente que fluye satisface la ecuación diferencial, (desarrollo matemático en [1, p. 273-274]):

$$RC \frac{dV_C}{dt} + V_C = V_0 \quad (1)$$

cuya solución para condiciones iniciales  $V_C(0) = 0$  es:

$$V_C(t) = V_0 (1 - e^{-t/\tau}) \quad (2)$$

donde  $\tau = RC$  es la constante de tiempo del circuito. Cuando la fuente se desconecta (descarga), la solución es:

$$V_C(t) = V_0 e^{-t/\tau} \quad (3)$$

El condensador se considera completamente cargado (o descargado) tras aproximadamente  $5\tau$ .

### II-B. Conversión analógico a digital

El ADC de 16 bits utilizado en la FPGA convierte la tensión analógica  $V_C$  en un valor entero  $N$  en el rango  $[0, 2^{16} - 1] = [0, 65535]$ . El voltaje normalizado se obtiene como:

$$V_{norm} = \frac{N}{65535} \quad (4)$$

Esta normalización permite comparar directamente la curva experimental con la predicción teórica expresada en unidades de  $V/V_{max}$ .

### III. METODOLOGÍA

#### III-A. Descripción del circuito

El circuito consiste en una resistencia  $R$  en serie con un condensador  $C$ , conectados a la salida de un generador de onda cuadrada implementado en la FPGA donde el canal de entrada del ADC mide la tensión del condensador.

Los valores de los elementos del circuito son:

- Resistencia:  $R = 10\text{ k}\Omega$
- Condensador:  $C = 10\text{ }\mu\text{F}$
- Constante de tiempo teórica:  $\tau = RC = 0,1\text{ s}$
- Frecuencia de muestreo configurada en la FPGA: registro 0x6, equivalente a 1kHz según nuestro diseño en la FPGA.
- Número de muestras adquiridas:  $N = 1024$

#### III-B. Montaje experimental

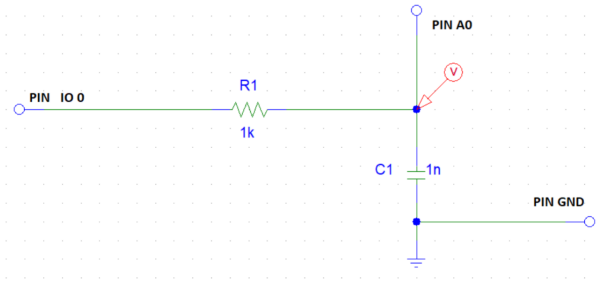


Figura 1: Circuito del montaje experimental. FPGA PYNQ-Z1 con el circuito RC ( $R = 10\text{ k}\Omega$ ,  $C = 10\text{ }\mu\text{F}$ ) conectado al canal ADC.

La tarjeta PYNQ-Z1 se configuró cargando el bitstream `cargaDescarga.bit`, que implementa en la FPGA un módulo de control de frecuencia mediante GPIO y un bloque DMA para la transferencia de datos al procesador ARM, el circuito RC externo se conectó al pin de salida del generador de onda cuadrada y al canal de entrada del ADC.

#### III-C. Adquisición de datos

El proceso de adquisición se realizó con el script Python `fpga-capacitor-data-retrieval.py`. Los pasos ejecutados fueron:

1. Se carga el bitstream sobre la FPGA con `Overlay`.
2. Se configura la frecuencia de la señal de excitación mediante el GPIO (`frequency = 0x6`).
3. Se espera un tiempo de adquisición de 3 segundos.
4. Se aloja un buffer de 1024 muestras de tipo `uint32` y se inicia la transferencia DMA.
5. Los datos se normalizan dividiendo entre 65535 (Ec. 4) y se guardan en un archivo CSV.

#### III-D. Procesamiento de datos

El script `fpga-charge-discharge-curve.py` lee el CSV generado, aplica nuevamente la normalización y produce la gráfica final de la curva de carga y descarga.

A continuación se presenta el código principal de adquisición:

```
from pynq import allocate, Overlay
import numpy as np

# Cargar bitstream
dma_dato = Overlay("cargaDescarga.bit")
dma = dma_dato.axi_dma
gp = dma_dato.axi_gpio_0

# Configurar frecuencia y esperar
gp.write(0x0, 0x6)
time.sleep(3)

# Transferencia por DMA
input_buffer = allocate(shape=(1024,), dtype=np.
                          uint32)
dma.recvchannel.transfer(input_buffer)

# Normalizacion
datos_norm = input_buffer / 65535
```

Listing 1: Adquisición de datos desde la FPGA (fragmento principal)

### IV. RESULTADOS

#### IV-A. Curva experimental

La Fig. 2 muestra la curva de voltaje normalizado en función del número de muestra obtenida con la FPGA.

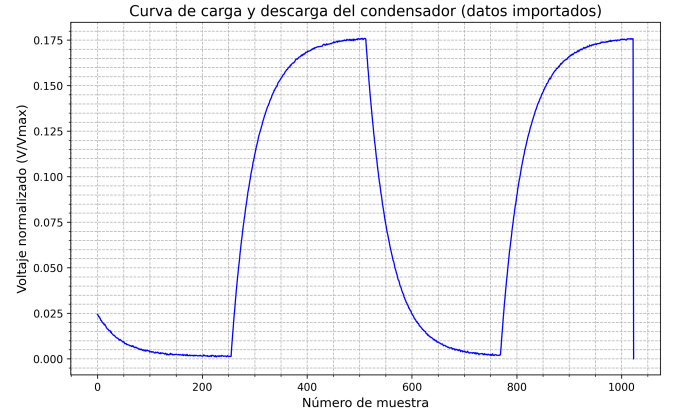


Figura 2: Curva de carga y descarga del condensador registrada por la FPGA. Eje vertical: voltaje normalizado  $V/V_{max}$ ; eje horizontal: número de muestra.

Se observan claramente dos ciclos completos de carga y descarga dentro de las 1024 muestras. La detección automática de flancos sobre la señal derivada identifica el inicio del primer ciclo de carga en la muestra 256 y el inicio de la descarga en la muestra 513, con un semiciclo de carga de  $\Delta n_{carga} = 513 - 256 = 257$  muestras. El segundo ciclo es prácticamente idéntico: carga desde la muestra 770 hasta la muestra 1023 ( $\Delta n_{carga} = 253$  muestras). El voltaje máximo normalizado alcanzado experimentalmente es:

$$V_{norm, max} \approx 0,176 \frac{V}{V_{max}} \quad (5)$$

Esto indica que la tensión de alimentación aplicada por la FPGA es aproximadamente el 17.6 % del fondo de escala del ADC (1 V), es decir,  $V_0 \approx 0,176$  V.

#### IV-B. Detección automática de transiciones

Para determinar con precisión las muestras de inicio de carga y descarga se calculó la derivada discreta de la señal normalizada  $v[n]$ :

$$\Delta v[n] = v[n] - v[n-1] \quad (6)$$

Se definieron dos umbrales sobre  $\Delta v[n]$ : uno positivo  $\delta^+ = 0,002$  para detectar el flanco de subida (inicio de carga) y uno negativo  $\delta^- = -0,002$  para el flanco de bajada (inicio de descarga). Un evento se registra en el índice  $n$  cuando  $\Delta v[n] > \delta^+$  o  $\Delta v[n] < \delta^-$ , imponiendo además una separación mínima de 30 muestras entre eventos consecutivos para evitar detecciones múltiples dentro del mismo bloque. La Fig. 3 muestra la curva con las transiciones identificadas.

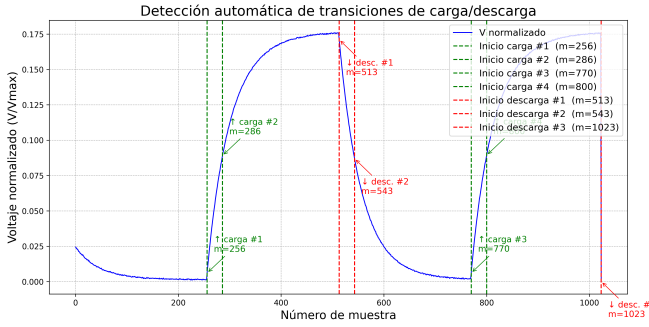


Figura 3: Transiciones de carga (verde) y descarga (rojo) detectadas automáticamente mediante umbrales de la derivada discreta.

Los eventos etiquetados como carga #2 y descarga #2 corresponden a detecciones intermedias dentro del mismo flanco, producto del umbral seleccionado; no representan ciclos independientes y se excluyen del análisis cuantitativo. Lo mismo aplica para carga #4, cuyo flanco queda truncado al final del buffer.

#### IV-C. Estimación de la constante de tiempo

El valor de voltaje correspondiente a  $t = \tau$  es:

$$V_C(\tau) = V_0 (1 - e^{-1}) \approx 0,632 \times 0,176 \approx 0,111 \frac{V}{V_{max}} \quad (7)$$

A partir de los datos experimentales, dicho nivel se alcanza en la muestra 300, siendo el inicio de carga la muestra 256. La constante de tiempo experimental en muestras es por tanto:

$$\tau_{exp} = 300 - 256 = 44 \text{ muestras} \quad (8)$$

## V. ANÁLISIS Y COMPARACIÓN CON LA TEORÍA

### V-A. Forma de la curva

La curva obtenida (Fig. 2) concuerda cualitativamente con las ecuaciones (2) y (3). La fase de carga muestra un crecimiento exponencial que satura en  $V_0$ , mientras que la descarga exhibe un decaimiento exponencial hacia cero, ambos con la misma constante de tiempo, como predice la teoría.

### V-B. Comparación cuantitativa

Con los valores del circuito ( $R = 10 \text{ k}\Omega$ ,  $C = 10 \mu\text{F}$ ) la constante de tiempo teórica es:

$$\tau_{teo} = RC = (10 \times 10^3)(10 \times 10^{-6}) = 0,1 \text{ s} \quad (9)$$

La frecuencia de muestreo fue configurada mediante el registro 0x6, lo que corresponde a  $f_s = 1 \text{ kHz}$  según la tabla de frecuencias del módulo:

$$f_s = 1000 \text{ muestras/s} \quad (10)$$

Con esto, la constante de tiempo experimental en segundos es:

$$\tau_{exp} = \frac{44 \text{ muestras}}{1000 \text{ muestras/s}} = 0,044 \text{ s} \quad (11)$$

El error relativo respecto al valor teórico es:

$$\varepsilon = \frac{|\tau_{exp} - \tau_{teo}|}{\tau_{teo}} \times 100 \% = \frac{|0,044 - 0,1|}{0,1} \times 100 \% = 56 \% \quad (12)$$

Esta discrepancia es atribuible a dos factores principales: en primer lugar, la tolerancia del condensador utilizado (típicamente  $\pm 20 \%$ ) y de la resistencia ( $\pm 5 \%$ ) pueden desplazar el valor real de  $\tau_{teo}$  entre 0,075 s y 0,125 s; en segundo lugar, la estimación de  $\tau_{exp}$  se obtiene identificando la primera muestra discreta donde  $V \geq 0,632 V_0$ , lo que introduce un error de cuantización de hasta  $\pm 1/f_s = 1 \text{ ms}$ . No obstante, un error del 56 % supera lo esperado por tolerancias de componentes, lo que sugiere que la frecuencia de muestreo efectiva del bitstream podría diferir de 1 kHz nominal, o que los valores de  $R$  y  $C$  difieren significativamente de los reales.

### V-C. Discusión

A pesar de la discrepancia numérica en  $\tau$ , la forma exponencial de la curva corrobora el modelo teórico descrito por las Ecs. (2) y (3), el comportamiento cualitativo es correcto: la carga y la descarga son asimétricas en amplitud (el condensador no alcanza  $V_{max}$  del ADC porque  $V_0 \ll V_{ADC,max}$ ), pero simétricas en forma temporal dentro de cada semiciclo.

El error relativo obtenido ( $\varepsilon = 56 \%$ ) supera lo atribuible únicamente a las tolerancias de los componentes pasivos ( $\pm 5 \%$  en  $R$  y  $\pm 20 \%$  en  $C$ ). Una posible causa es que la frecuencia de muestreo efectiva del bitstream no coincida exactamente con el valor nominal de 1 kHz del registro 0x6; otra es que los valores reales de  $R$  y  $C$  se aparten de sus nominales, algo esperable en resistencias de carbono y condensadores electrolíticos sin medición previa. Para reducir

la incertidumbre convendría medir  $R$  y  $C$  directamente con un multímetro, verificar la frecuencia efectiva del bitstream, y estimar  $\tau$  mediante un ajuste exponencial por mínimos cuadrados en lugar de una búsqueda por umbral.

## VI. CONCLUSIONES

Se logró adquirir exitosamente la curva de carga y descarga de un condensador usando una FPGA PYNQ-Z1, demostrando la viabilidad de estas plataformas para experimentos de laboratorio, así, los datos experimentales muestran el comportamiento exponencial predicho por la teoría de circuitos RC (Ecs. 2 y 3), con dos ciclos completos reproducibles dentro de las 1024 muestras adquiridas, y un voltaje máximo normalizado de  $V_{norm} \approx 0,176$ , correspondiente a la tensión de salida de la FPGA, significativamente menor al fondo de escala del ADC (1 V).

La detección automática de transiciones mediante umbrales de la derivada discreta permitió identificar con precisión de muestra los instantes de inicio de carga (muestras 256 y 770) y de descarga (muestras 513 y 1023), y a partir de estos la constante de tiempo experimental resultó  $\tau_{exp} = 0,044$  s, con un error relativo del 56 % respecto al valor teórico  $\tau_{teo} = RC = 0,1$  s, atribuible a la incertidumbre en la frecuencia de muestreo real del bitstream y a las tolerancias de los componentes pasivos, también, el uso de Python junto con la librería `pynq` simplificó considerablemente la interfaz entre el hardware FPGA y el análisis de datos, permitiendo automatizar el procesamiento y la detección de eventos sobre la señal adquirida.

## REFERENCIAS

- [1] W. H. Hayt, J. E. Kemmerly, and S. M. Durbin, *Engineering Circuit Analysis*, 8th ed. New York: McGraw-Hill, 2012.
- [2] AMD Xilinx, “Zynq-7000 soc technical reference manual (ug585),” <https://docs.xilinx.com/r/en-US/ug585-zynq-7000-SoC-TRM>, 2023, accedido: junio 2025.
- [3] Xilinx, Inc., “PYNQ: Python productivity for zynq,” <http://www.pynq.io/>, 2023, accedido: junio 2025.